# Parkspot

Software Requirements Specification

November 7, 2025

Group DIAS

Andres Sanchez

Israel Osunkoya

Sara Mosquera

Daniel Quach

Submitted in partial fulfillment

of the requirements of Comp 3059 Computer Programing and Analysis

# Table of Contents

**1.0 Introduction ..... 3**
1.1 Purpose ..... 3
1.2 Scope ..... 3
1.3 References ..... 5
2.0 System Overview ..... 6
2.1 Project Perspective ..... 6
2.2 System Context ..... 7
2.3 General Constraints ..... 8
2.4 Assumptions and Dependencies ..... 9
3.0 Functional Requirements ..... 9
3.1 Functional Requirement or Feature #1 ..... 9
3.2 Use Cases ..... 10
3.2.1 Use Case 1: Search and Reserve Parking Spot ..... 10
3.2.2 Use Case 2: Manage Parking Spot Listing ..... 12
3.3 Data Modelling and Analysis ..... 13
3.4 Process Modelling ..... 18
4.0 Non-Functional Requirements ..... 19
5.0 Logical Database Requirements (LDR) ..... 21
6.0 Other Requirements ..... 24
**7.0 Approval ..... 24**

# 1.0 Introduction

ParkSpot is a mobile platform designed to address the urban challenges of parking scarcity by connecting drivers who need short-term parking with property owners who have unused parking spaces . By allowing owners to list and rent out their unused spots and allowing drivers to search, reserve, and pay for parking in real time, ParkSpot creates a community-driven marketplace that reduces search time, traffic congestion, and generates passive income for owners.

## 1.1 Purpose

This Software Requirements Analysis and Design document describes the high-level functional and non-functional requirements for the ParkSpot system. It defines what the system must do to meet the needs of its primary users, which are drivers and parking space owners, without specifying how the solution will be technically implemented. This document serves as a central reference for us, the development team (Group DIAS), course instructors, and any future stakeholders to ensure all development efforts remain aligned with the agreed project objectives. It will guide future design, implementation, and testing phases of the capstone project.

## 1.2 Scope

The ParkSpot system will be delivered as a **Minimum Viable Prototype (MVP)** of a mobile platform with the following core capabilities:

- User authentication for two primary roles: **Drivers** and **Parking Space Owners**

- Ability for owners to create, edit, and manage parking spot listings (location, price, availability windows)
- Search and filtering functionality for drivers to locate available spots by destination, price, distance, and time
- Reservation and secure payment processing
- Basic GPS navigation to reserved spots using integrated mapping services
- Notification system for reservation expiry, extension options, and overstay alerts
- Automated grace period (15 minutes) and penalty charge handling
- Admin dashboard for dispute resolution, listing approval, and basic analytics

**Benefits**

- Reduces time and fuel wasted searching for parking
- Monetizes unused private parking assets
- Decreases urban congestion caused by roaming vehicles (who can't find parking)
- Provides a safe alternative to cash-based or informal parking arrangements

**Out of Scope**

- Full-scale commercial launch or marketing
- Native iOS application development(for now)
- Advanced mapping features beyond basic navigation (e.g, real-time traffic layers, 3D views, AI incorporation)
- Physical towing logistics integration
- Multi-language support(for now)
- Production-level payment processing (only test/sandbox mode will be used)

## 1.3 References

| **Title** | **Source** |
|---|---|
| Project Summary | F25_T29_Project Summary.docx |
| Project Vision | F25_T29_Project Vision.docx |
| High-Level Requirements | F25_T29_High Level Requirements.docx |
| Team Charter | F25-Team Charter.docx |
| Project Plan | F25-Project Plan Template.docx |

# 2.0 System Overview

The System Overview section introduces the system context and design.

## 2.1 Project Perspective

ParkSpot is a new, self-contained mobile and web-based system developed by group DIAS for Capstone Project. It is a digital solution that addresses the lack of accessible parking, and looks to be community driven.

The key subsystems are:

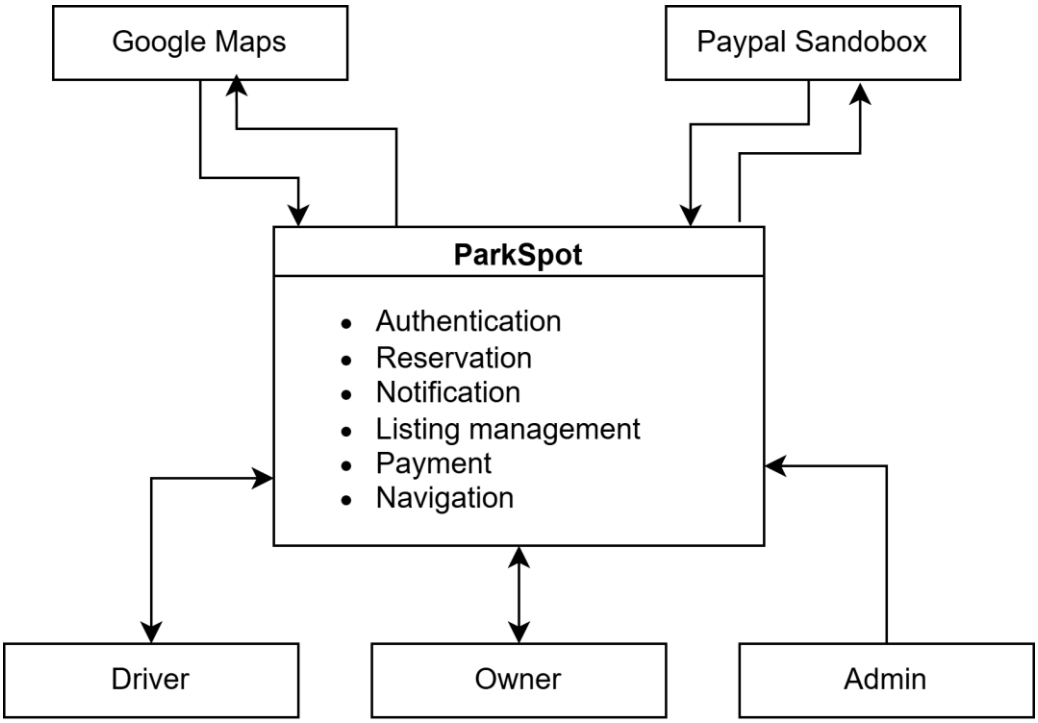
- **Authentication and user management:** Handles registration, login and role-based access for drivers, owners and admins.
- **SSListing management:** Allows Owners to create, edit and deactivate parking spot listings with price, address and availability windows.
- **Search and Reservations:** Enables Drivers to search for nearby spots, filter by price, time or distance and reserve in real time.
- **Payment Gateway integration:** Processes transactions securely via a third-party.
- **Navigation and Notifications:** Provides confirmation, map routing to the reserved spot and expiry alerts.
- **Admin Dashboard:** Supports the user and is used to monitor the listings and basic analytics.

All subsystems communicate through an API, backed by a relational Database. The architecture is service-oriented and containerized by Docker to simplify the deployment and keep the consistency in all environments.

## 2.2 System Context

The ParkSpot system interacts with multiple external entities:

| **External Entity** | **Interaction with System** |
|---|---|
| Driver | Registers / Login, searches for available parking, reserves and pays for spots, receives notifications and navigation links. |
| Owner | Lists and manages available spots, defines pricing and availability, views earnings and book history. |
| Admin | Monitors listings and bookings, manages disputes and reviews the system through a read-only dashboard. |
| Payment Gateway | Handles payment processing. |
| GPS provider | Provides navigation and distance calculator for reservations. |




## 2.3 General Constraints

- **Time Constraint:** The MVP must be completed between september 12 2025 and April 1 2026.
- **Resource Constraint:** Development is limited to four student developers working part-time.
- **Budget Constraint:** No paid hosting or payment services. Free or student-licensed tools will be used.
- **Technical Constraint:** only mobile app.
- **Security constraint:** Payment handling limited to test environments.
- **Data Constraint:** Personal and transaction data must be stored securely using encryption and hashed credentials.

## 2.4 Assumptions and Dependencies

**Assumptions**

- Users have internet connection and GPS enabled devices.
- Third party APIs remain available and functional.
- The MVP environment will operate in testing configuration.

**Dependencies**

- Firebase Authentication and Firestore/Realtime Database.
- Stripe SDK for Android.
- Google Maps SDK for navigation.
- Android Jetpack components (ViewModel, LiveData).
- Gradle build system and Android Studio.

# 3.0 Functional Requirements

This section describes specific features of the software project.

### 3.1 Functional Requirement or Feature #1

Spot Search and Booking (Drivers)

**Introduction**

- This feature allows drivers to search for and reserve parking spots in nearby locations

**Input**

- Search criteria such as location, date/time, and duration
- Selected parking spot for booking

**Processing**

- Query the database to find available spots matching criteria
- Calculate total cost based on time duration and price
- Generate a booking record and update the spot's availability

**Outputs**

- Display a list of available spots
- Confirmation page showing booking details and cost
- Booking confirmation message or email.

## 3.2 Use Cases

This section defines the boundary and behavior of the system by describing the goals of the primary actors (Driver, Space Owner) in scenarios derived from the High-Level Requirements (HLR) and System Features (SF).

### 3.2.1 Use Case 1: Search and Reserve Parking Spot

| Field | Description |
|---|---|
| **Use Case ID** | UC-001 |

| | |
|---|---|
| **Use Case Name** | Search and Reserve Parking Spot |
| **Goal** | Allow a Driver to find, select, and successfully pay for a spot for a specific duration. |
| **Primary Actor** | Driver |
| **Supporting Requirements** | HLR03, HLR04, HLR05, HLR06, SF-3, SF-4 |
| **Pre-Conditions** | 1. The Driver must be logged into the system (SF-1). 2. The Driver has a valid payment method saved or enters one during checkout. |
| **Post-Conditions** | 1. A new RESERVATION record is created with a status of **Confirmed**. 2. The spot's availability is immediately updated in the database. 3. The Driver receives confirmation and navigation details. |

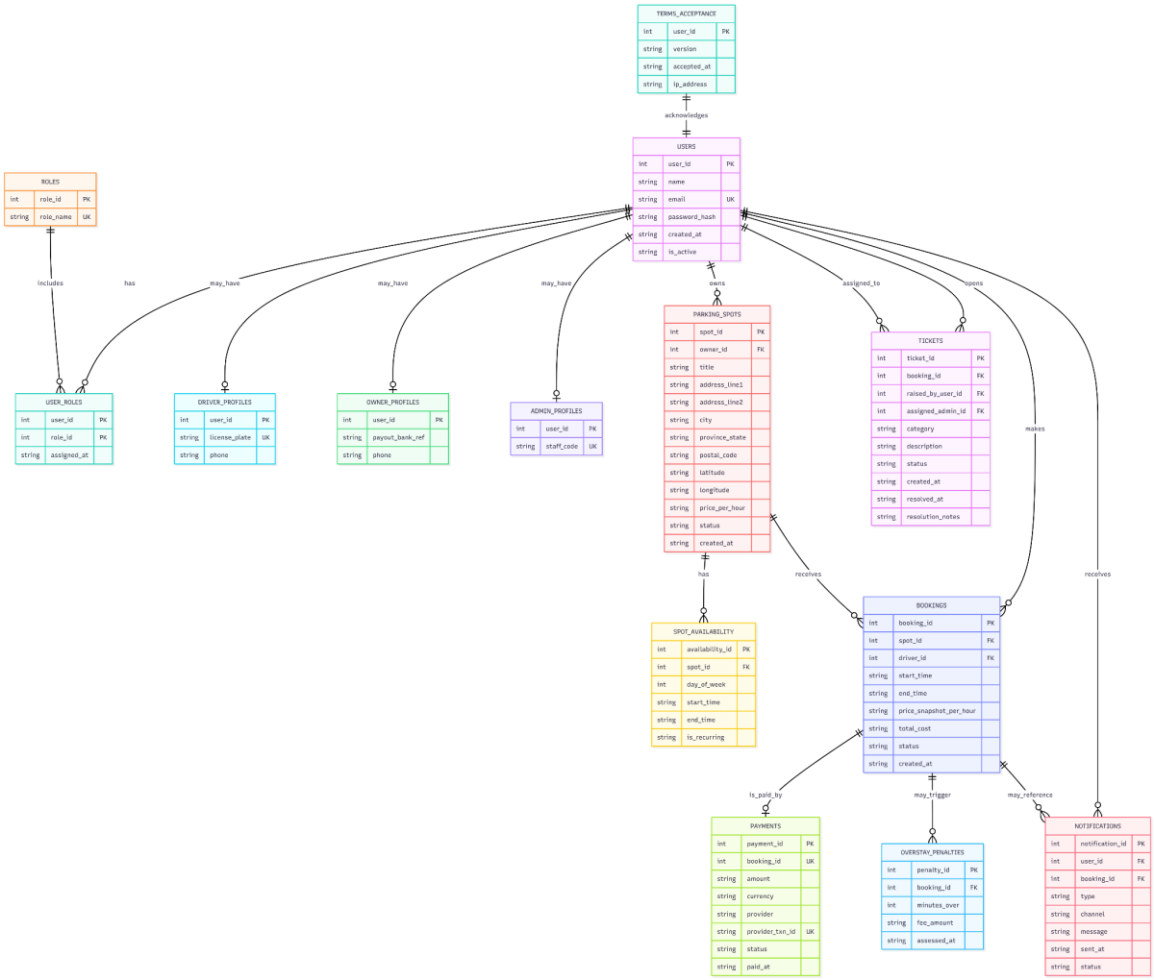
### 3.2.2 Use Case 2: Manage Parking Spot Listing

| Field | Description |
|---|---|
| **Use Case ID** | UC-002 |
| **Use Case Name** | Manage Parking Spot Listing |
| **Goal** | Allow a Space Owner to define the location, pricing, and availability schedule for their parking spot. |
| **Primary Actor** | Space Owner |
| **Supporting Requirements** | HLR01, HLR02, SF-2 |
| **Pre-Conditions** | 1. The Space Owner must be logged into the system (SF-1). 2. The Owner has a confirmed user profile. |

| | |
|---|---|
| **Post-Conditions** | 1. A new PARKING_SPOT record is created (or an existing one updated). 2. The spot is marked as **Active** and available for driver searches based on the defined schedule. |

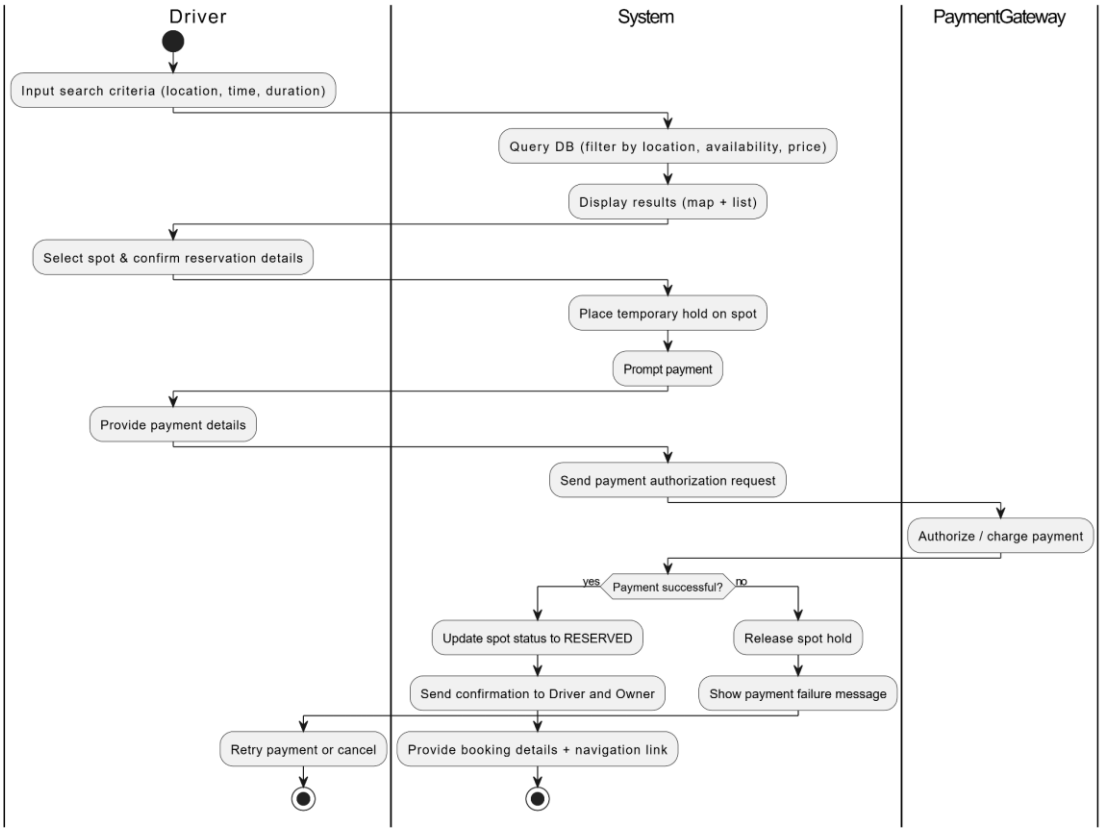
## 3.3 Data Modelling and Analysis

- Normalized Data Model Diagram

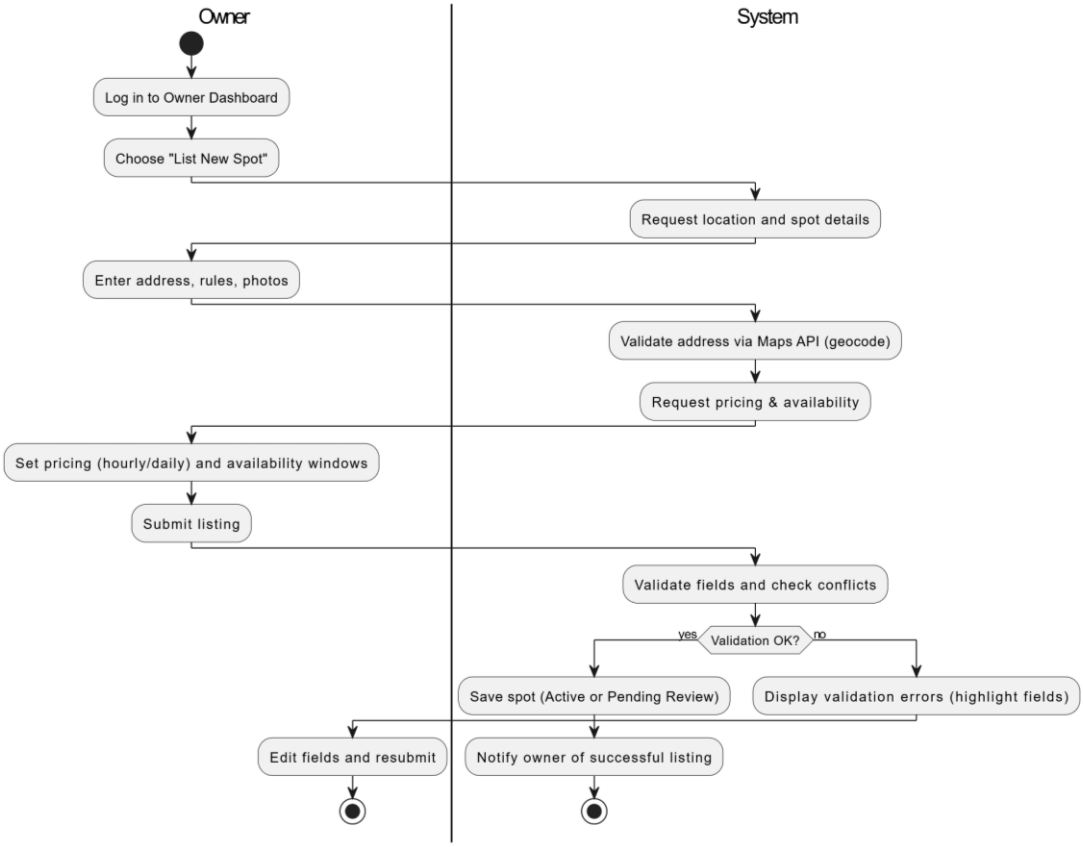


- Activity Diagrams

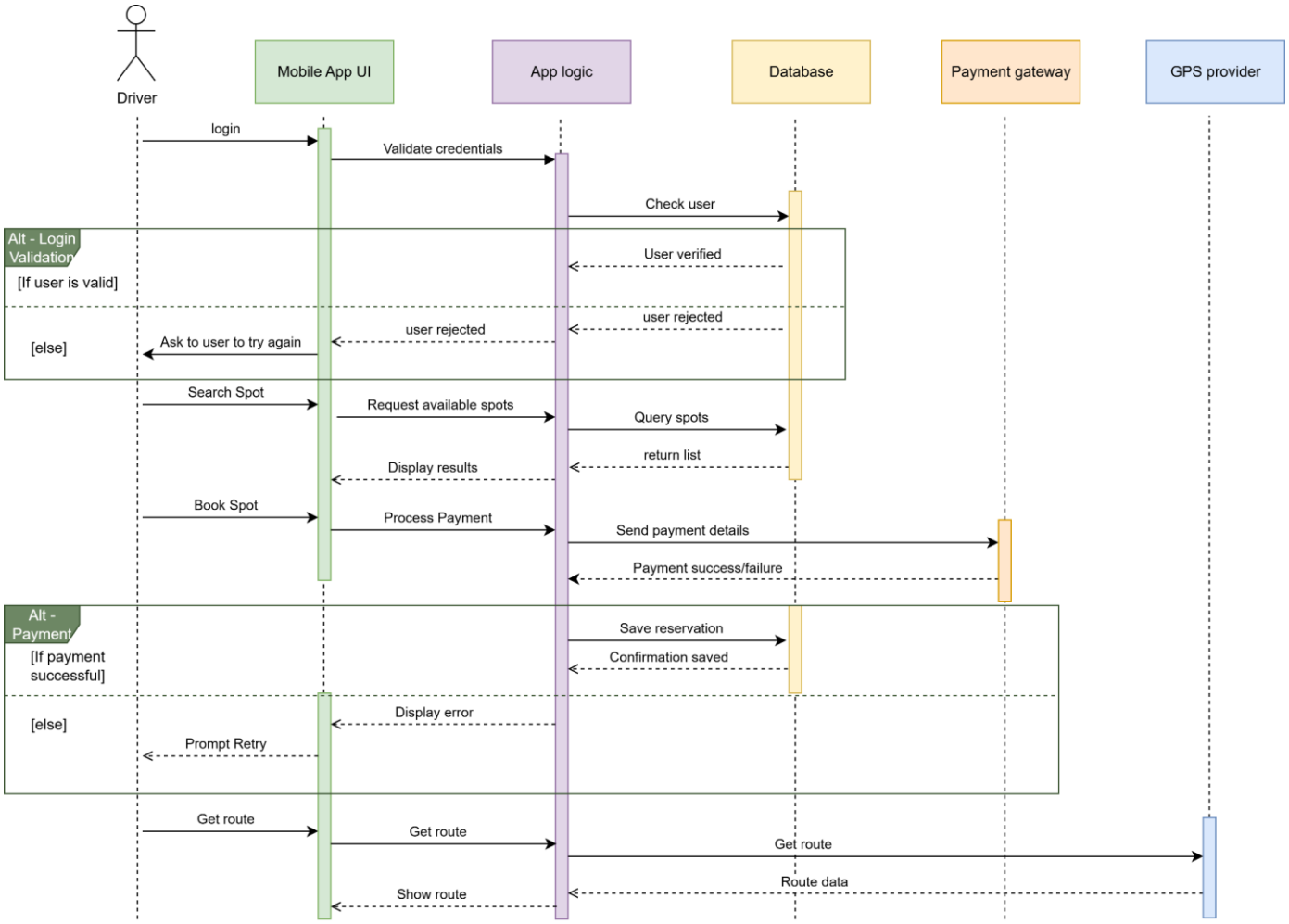
Driver Reserving a Spot:



Space Owner Listing a Spot:

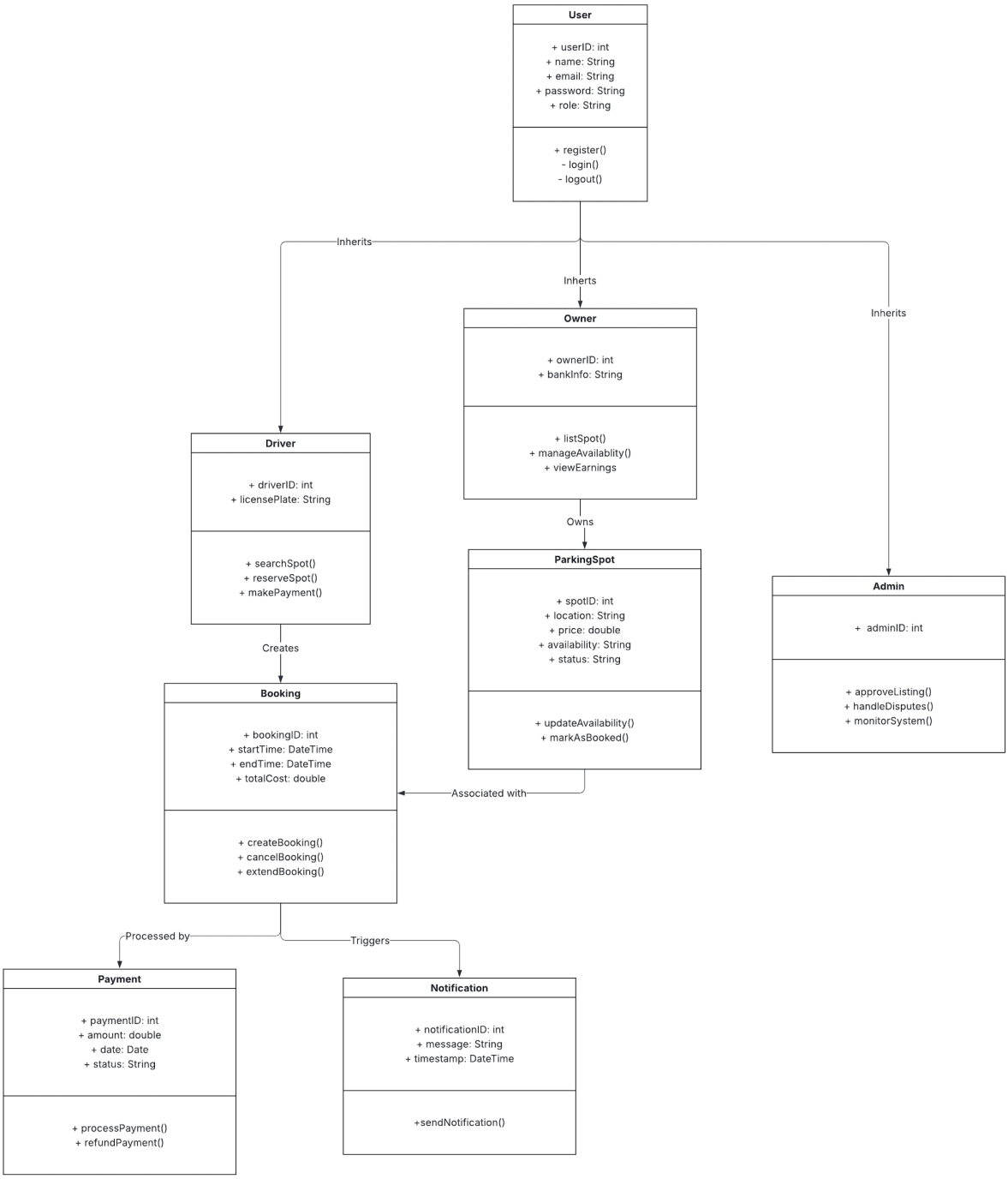


- Sequence Diagrams




Sequence Diagram: Driver booking a parking spot.

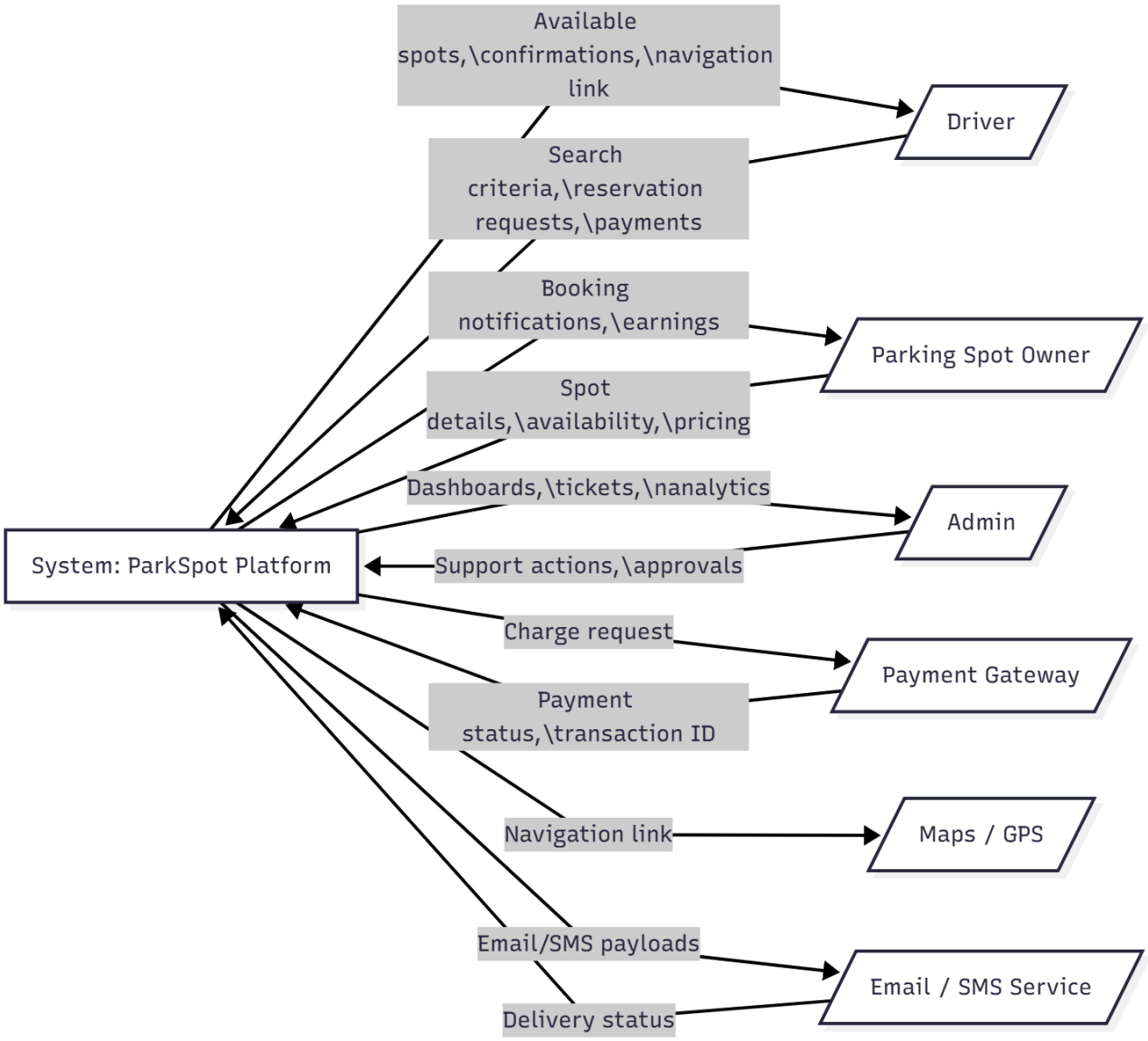
- UML Class Diagram



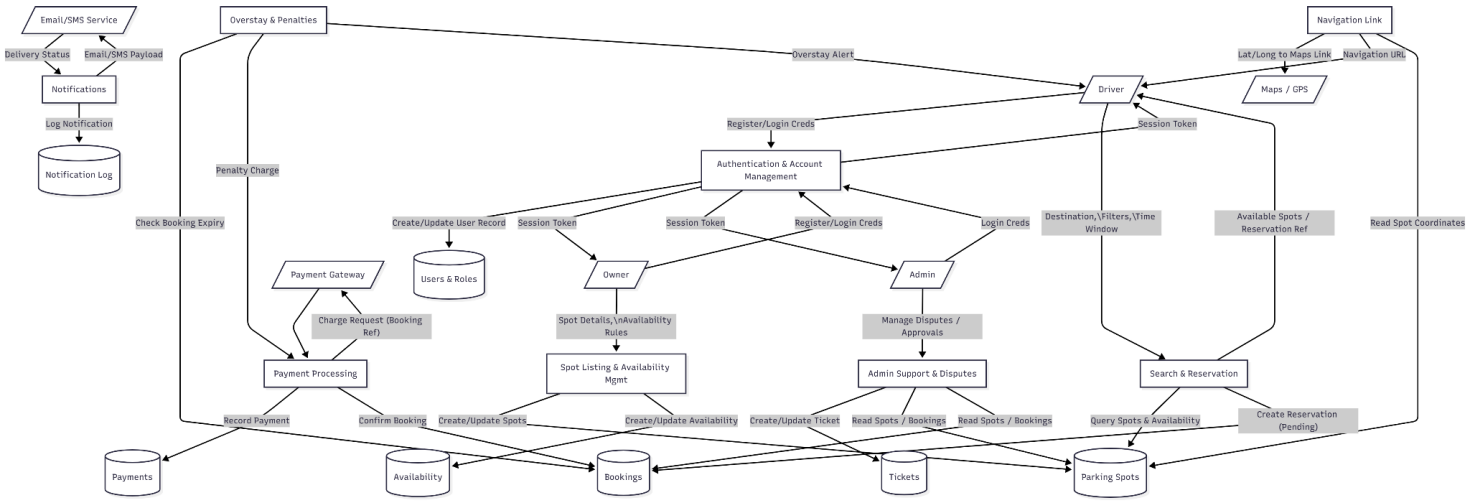
## 3.4 Process Modelling

- Data Flow Diagram

DFD Level 0 – Context Diagram



DFD Level 1 – Detailed View:




# 4.0 Non-Functional Requirements

### Performance Requirements

- The system shall process 95% of user transactions (reservations, payments, and cancellations) within 2 seconds.
- The application shall support at least 1000 concurrent users without noticeable performance degradation.
- The system shall display search results for available parking spots within 3 seconds after a query is made.

### Reliability Requirements

- The system shall have an error rate of less than 1% for completed transactions (e.g., payment, cancellation, and booking).

- The system shall automatically recover from server interruptions within 30 seconds.
- The system shall log all failed operations for review and debugging.

**Availability Requirements**

- The system shall have a minimum uptime of 95% per month, excluding scheduled maintenance.
- Scheduled maintenance shall not exceed 4 hours per month and will be announced at least 24 hours in advance.
- The app shall not notify users of system downtime or outages through an in-app message or email alert.

**Security Requirements**

- The system shall authenticate all users via secure login, with an optional two-factor authentication for added protection.
- All payment transactions shall be processed through a PCI-DSS compliant third-party payment gateway (e.g. Paypal, Stripe).
- Admin actions (e.g., dispute resolution, data export) shall be audited and logged for accountability.

**Maintainability Requirements**

- The system shall be modularized such that new features can be added or modified without affecting core functionalities.
- Maintenance updates shall be deployable with no more than 30 minutes of downtime.

- The system shall use containerization (e.g., Docker) to simplify deployment and dependency management.

**Portability Requirements**

- The application shall run and be compatible with Android and iOS
- All platform APIs shall be designed to allow future migration to cloud infrastructure with minimal code changes.

# 5.0 Logical Database Requirements (LDR)

A database is required to store user data, parking spot listings, real-time availability, reservation records, and transaction history. The logical design must support high concurrency, integrity, and security for financial and location data.

| Category | Requirement (LDR) | Detail / Constraints |
|---|---|---|
| **Data Formats** | **LDR-1: Geo-Coordinates** | The PARKING_SPOT entity must store location data as **Latitude and Longitude** (numerical floating-point format) to enable accurate map searching (HLR03) and navigation (HLR06). |

| | | |
|---|---|---|
| | **LDR-2: Time & Pricing** | All availability and reservation times must be stored in a **standard, localized time zone format** (e.g., UTC with offset), and all financial fields must be stored as **fixed-point decimals** to prevent rounding errors. |
| **Data Integrity** | **LDR-I1: Reservation Conflict** | The system must enforce a constraint to prevent two RESERVATION records from existing for the same PARKING_SPOT at overlapping times (HLR04). |
| | **LDR-I2: Referential Integrity** | Must ensure that a RESERVATION record always links to a valid USER (Driver) and PARKING_SPOT (Owner), maintaining strong relationships between entities. |
| **Storage Capabilities** | **LDR-S1: Password Security** | All USER passwords must be stored only as **hashed and salted values**; raw passwords are strictly prohibited (HLR09). |

| | | |
|---|---|---|
| | **LDR-S2: Payment Security** | The database **must not store** sensitive financial data (e.g., credit card numbers, CVVs). Only **masked tokens or unique transaction IDs** provided by the third-party payment gateway will be stored (HLR05). |
| **Data Retention** | **LDR-R1: Audit Trail** | A comprehensive audit trail (log) of all user actions (especially financial and security-related changes) must be retained for a minimum of **two years** for dispute resolution and legal compliance (SF-7). |
| | **LDR-R2: Historical Data** | All RESERVATION and TRANSACTION records must be retained indefinitely, with a plan for **archiving historical records** (older than one year) to a separate storage environment to ensure primary system performance. |

# 6.0 Other Requirements

No additional requirements.

# 7.0 Approval

The signatures below indicate their approval of the contents of this document.

| Project Role | Name | Signature | Date (dd/mm/yy) |
|---|---|---|---|
| Project Manager | Andres Sanchez | andres | 10/11/25 |
| Software Developer | Israel Osunkoya | Israel | 10/11/25 |
| Front-End Developer | Daniel Quach | DQ | 10/11/25 |
| Back-End Developer | Sara Mosquera | Sara | 10/11/25 |